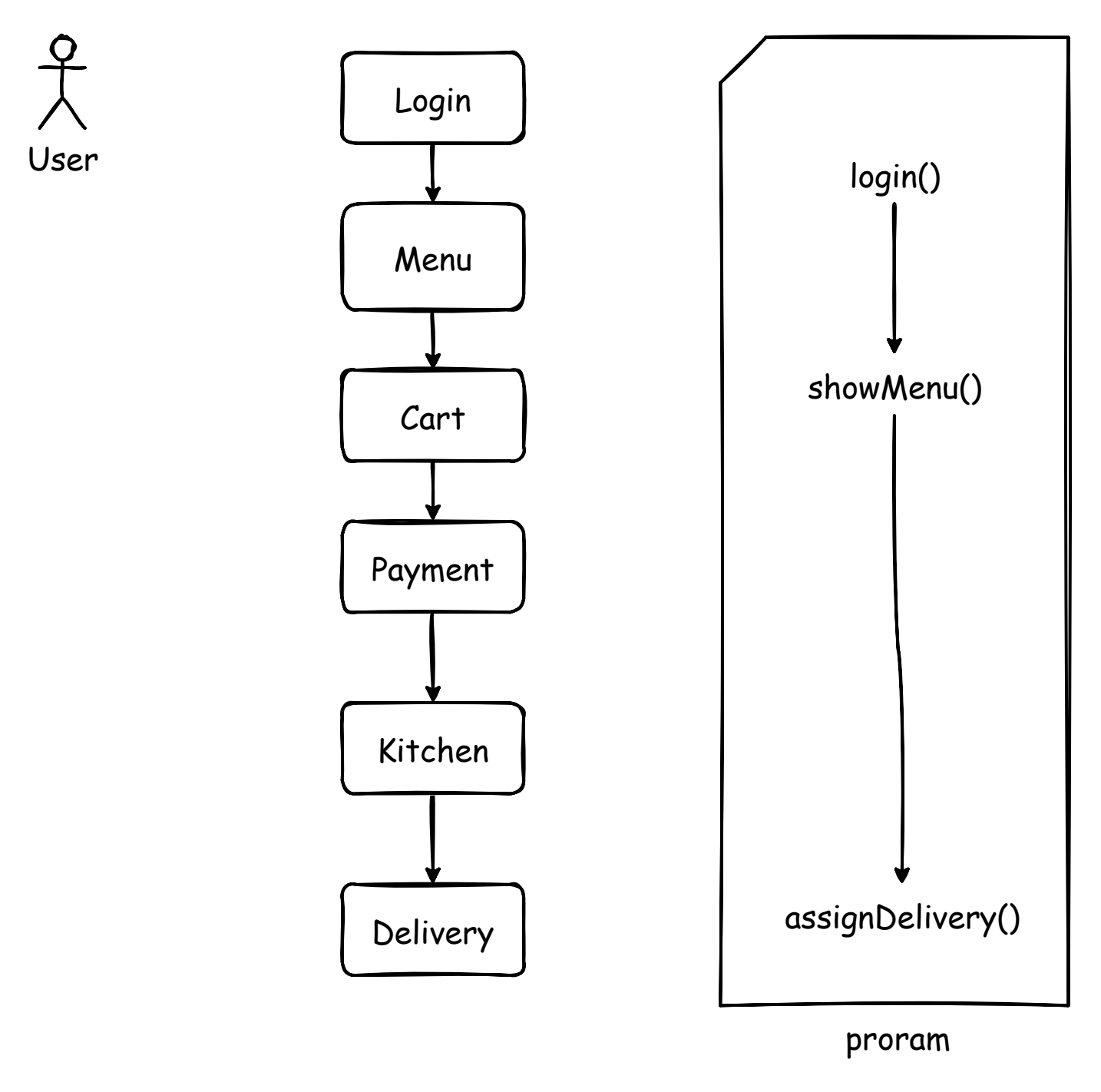
# Introduction to orchestration

Online Pizzaa Delivery system



Each function has exactly one responsibility.
↓
We write all this code in one project
↓
We compile it together.
↓
We test it together
↓
We package it together.

monolithic application

A monolithic application is an application in which multiple business capabilities are packaged and deployed as a single unit.

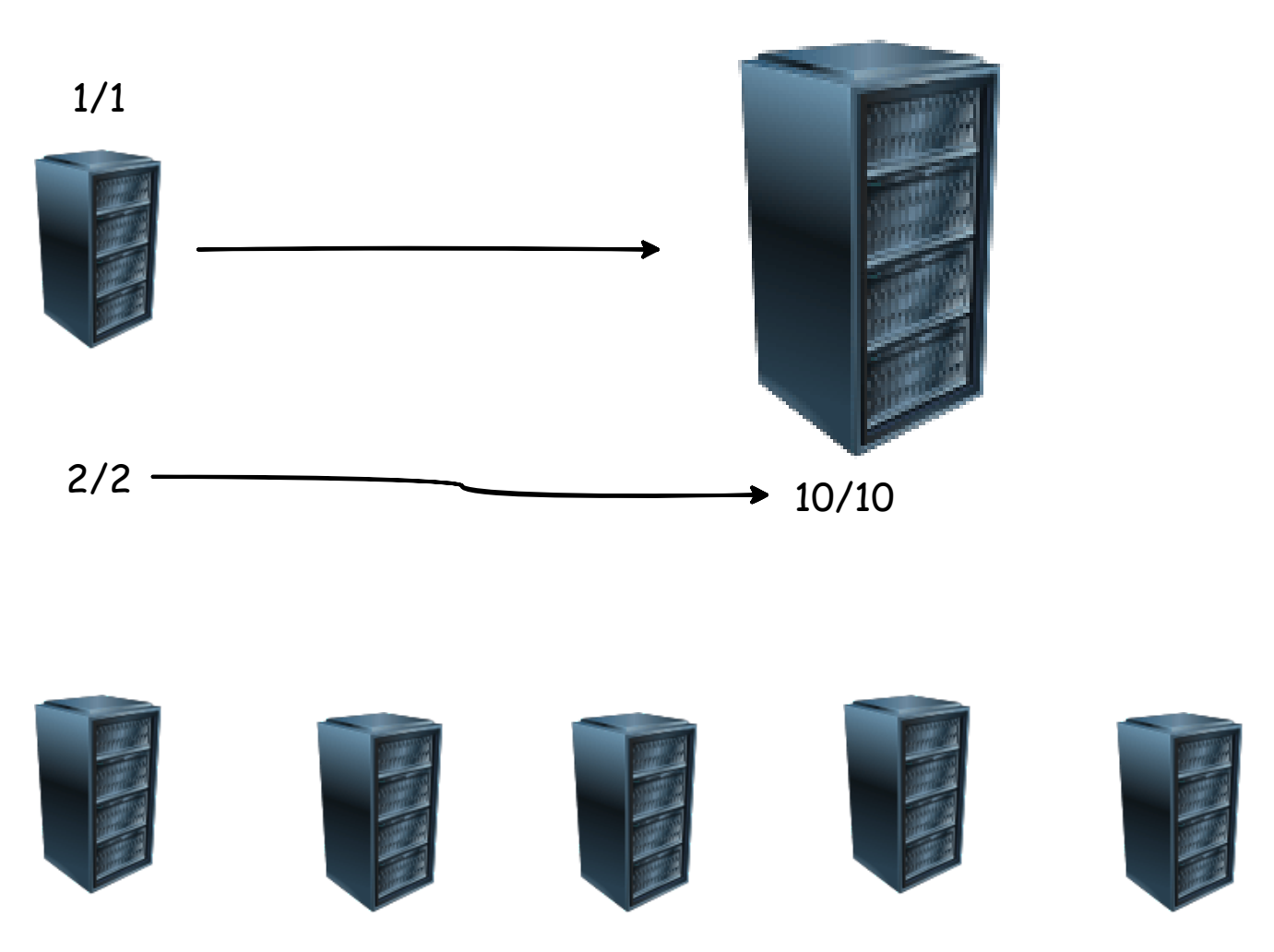
Simple architecture.
↓
Easy to develop for small teams.
↓
Simple debugging.

A hospital management system for a single hospital
↓
A payroll application

10 million users → 100 users

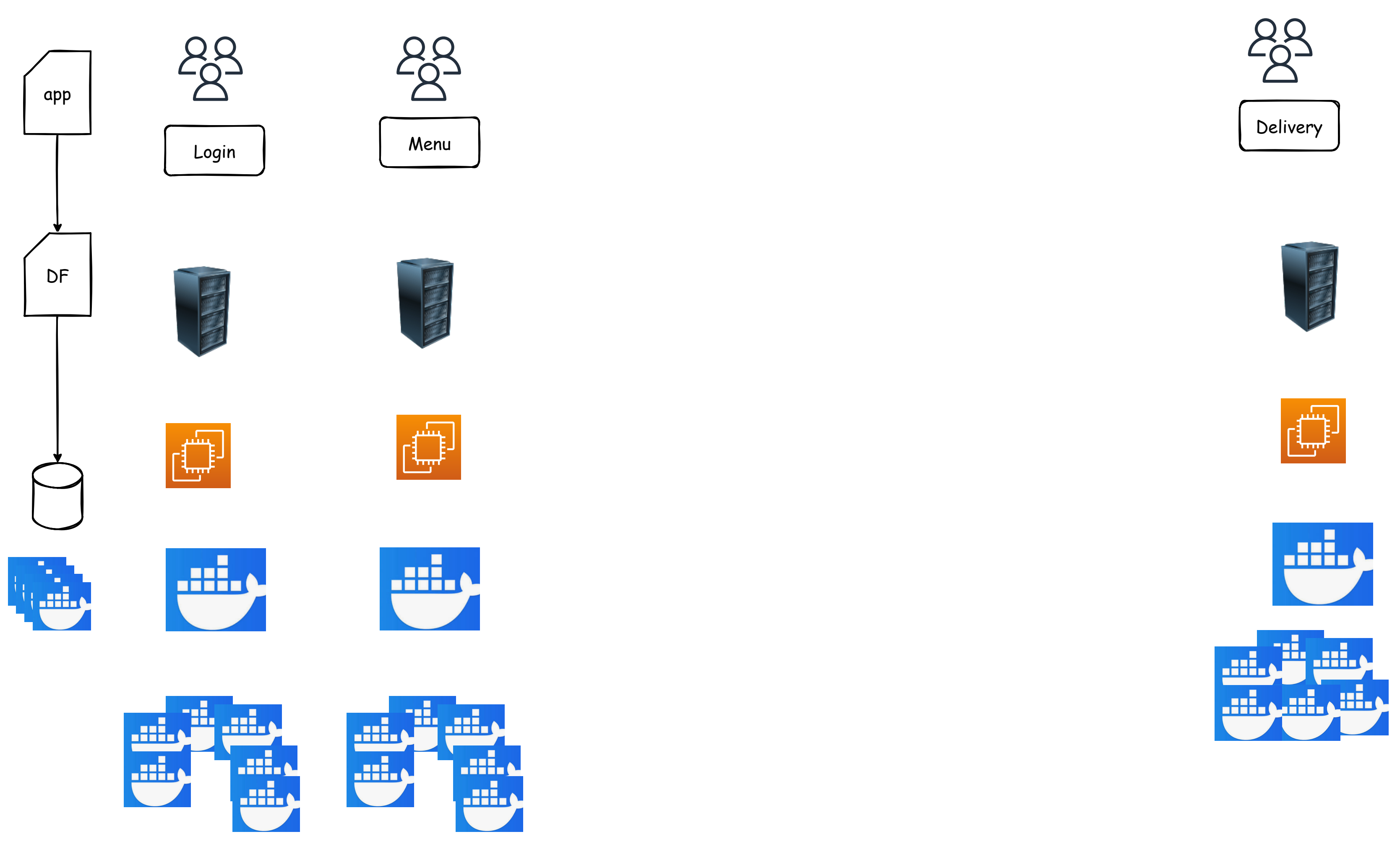
Login 5%
Menu 70%
Cart 10%
Payment 10%
Delivery 5%

Scaling



Because everything is packaged as one deployable unit, we must create another copy of the **entire application**,
even though only the Menu feature needs more capacity.

MicoServerices



Conatiner orchestration

Who starts all these containers?
↓
Who restarts them if they fail?
↓
Who decides which server should run them
↓
Who connects them together? → Who performs rolling updates? → Who performs load balancing?

Conatiner orchestration

docker
↓
docker run nginx
docker stop nginx
docker start nginx
docker rm nginx
→ This is called the Imperative approach.

Payment

Imperative
↓
Tell the system **HOW** to do it

Declarative
↓
Tell the system **WHAT** you want

Declarative approach.

I always want three containers of my Payment Service running.
↓
This is the Declarative approach.
↓
We tell the system **WHAT** we want.
↓
The system figures out **HOW** to achieve it.

**Container orchestration platforms are built around declarative thinking.**

Payment Service

Running Containers = 3 ← 2

## Desired State vs. Actual State

Desire State And Current State

instead of telling the orchestration platform *how* to do something, we simply tell it *what* we want.

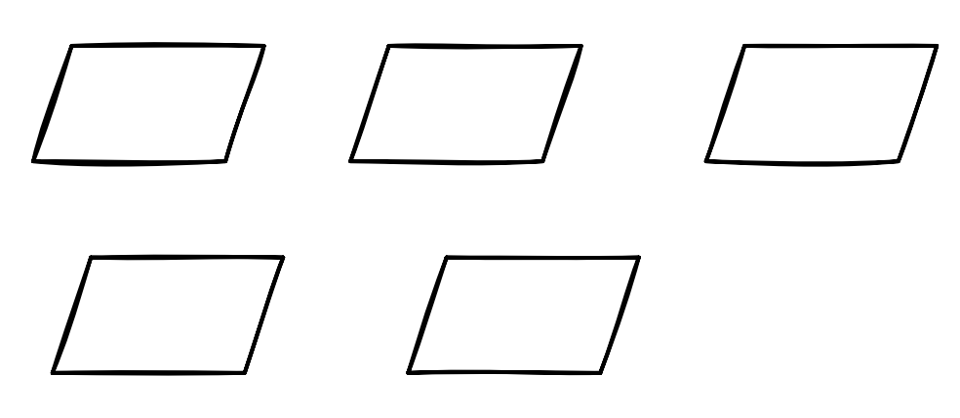
how does it know whether everything is still running correctly?

## Desired State

Desired State is the target configuration that you declare.
It represents how your application should always look.

DS == 3

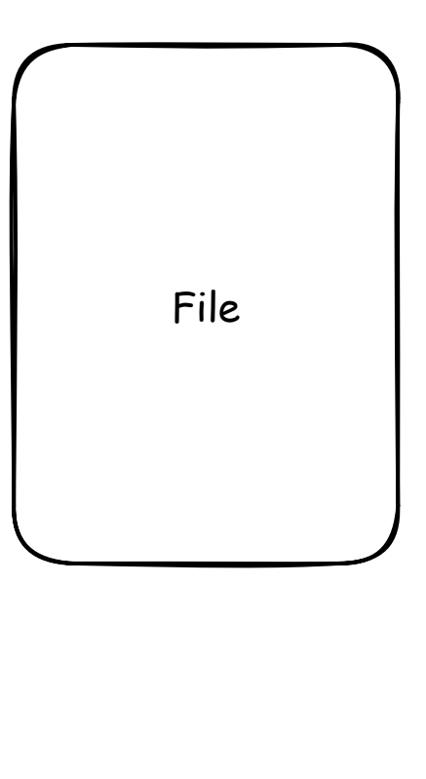
Actual State

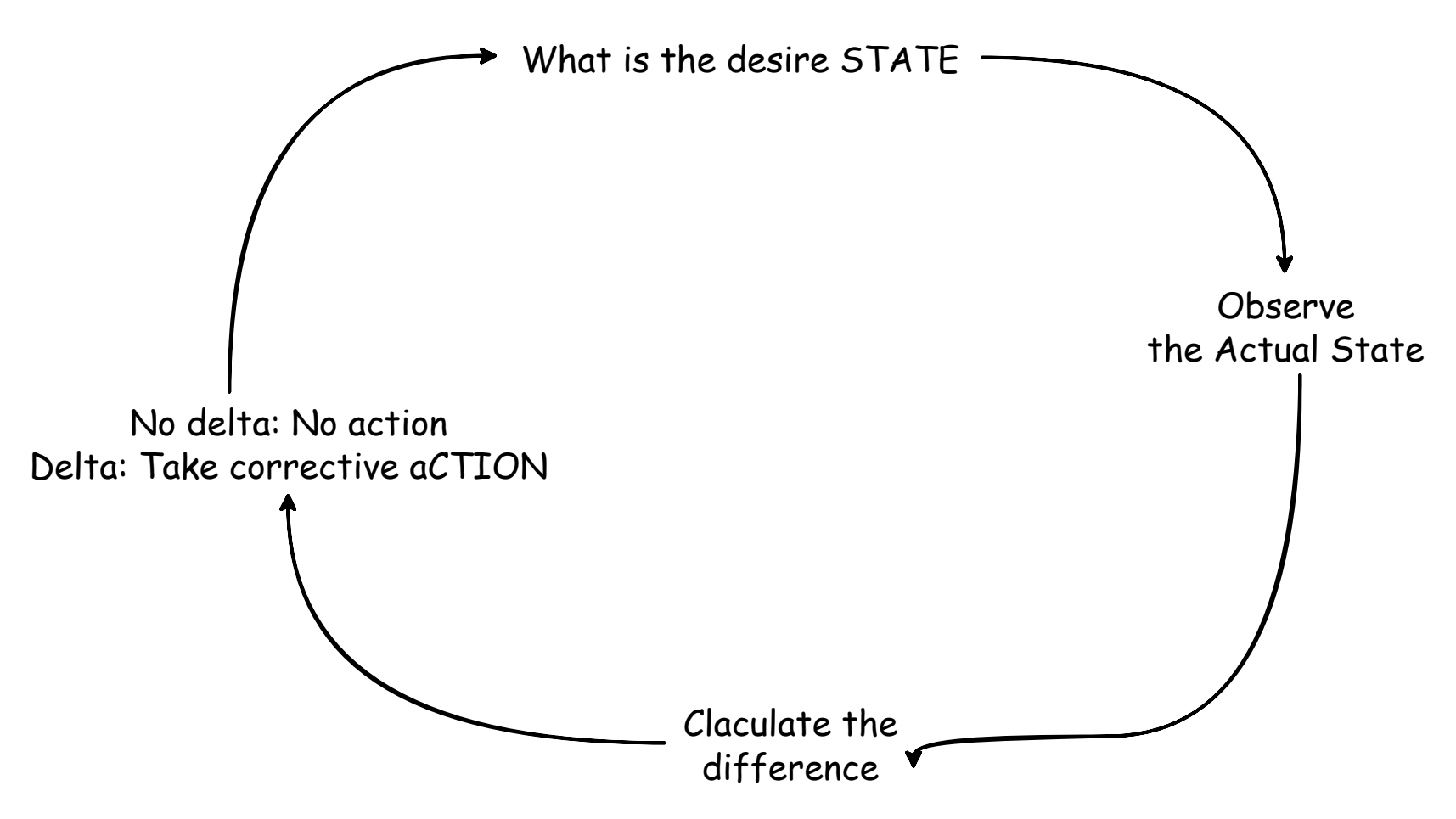


CS==2

**The entire job of a container orchestration platform is to continuously eliminate this difference.**

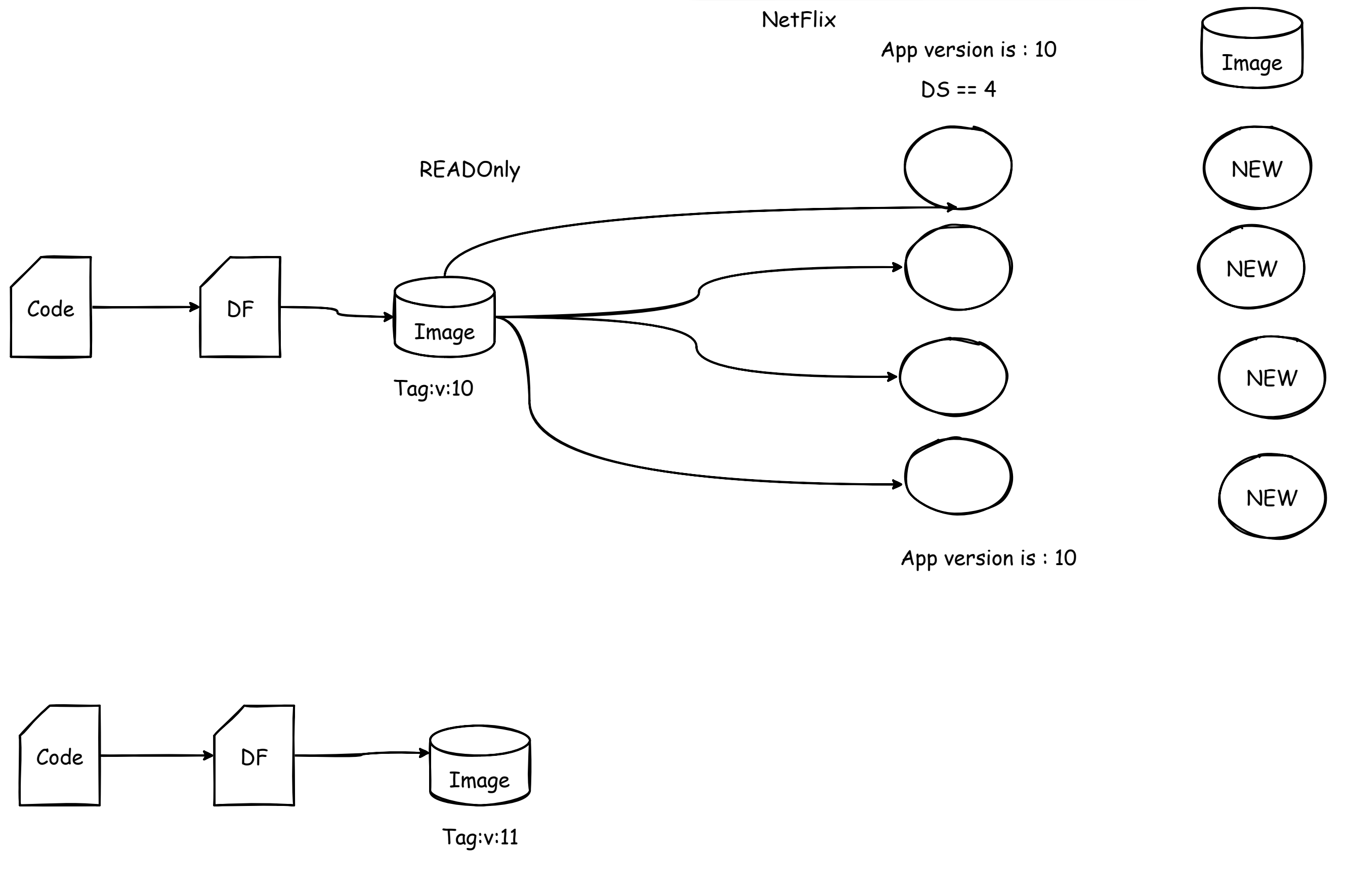
**Manual Operations**
↓
**Why they don't scale**
↓
**Container Orchestration**
↓
**Imperative vs Declarative**
↓
**Tell the platform WHAT you want**
↓
**Desired State**
↓
**Actual State**
↓
**There may be a gap**
↓
**Who keeps checking?**
↓
**Reconciliation Loop**
↓
**How does it fix failures?**
↓
**Self-Healing**

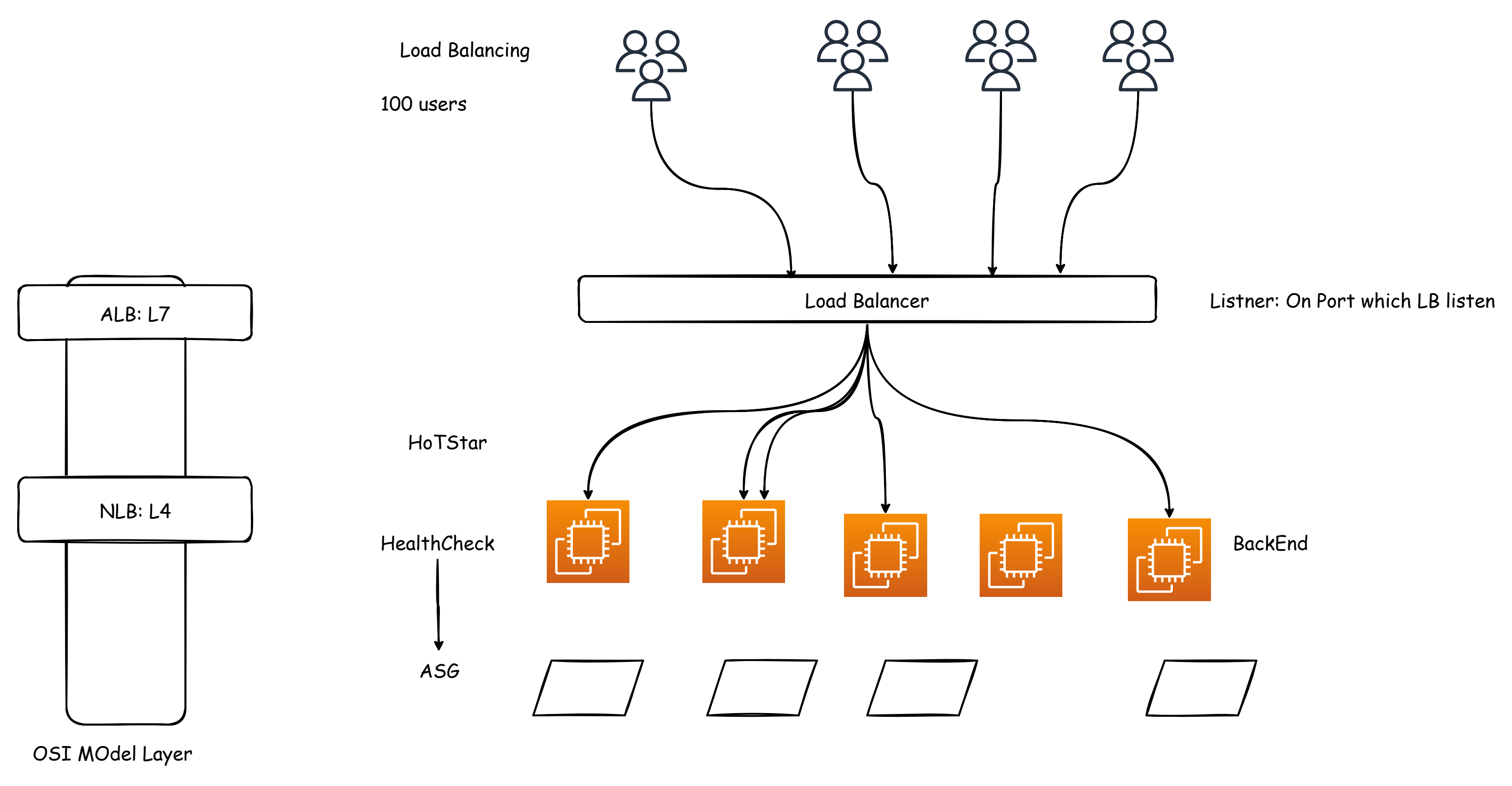




What is the desire STATE → Observe the Actual State → Claculate the difference → No delta: No action / Delta: Take corrective aCTION → What is the desire STATE

## Rolling Update





## Service Discovery

Pizza Application

Login → Menu → Cart → Payment → Kitchen → Delivery

It has to communicate with other services.

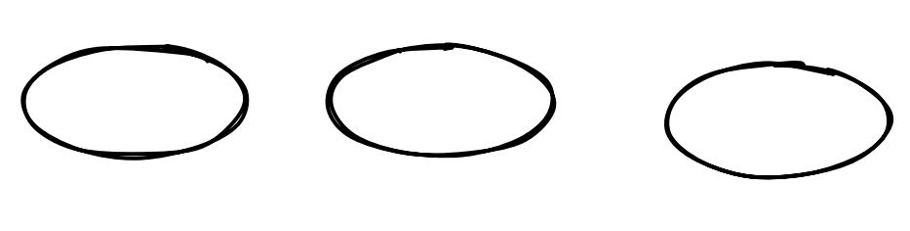
Cart ---------> Payment

Payment -------> Kitchen

Kitchen -------> Delivery

Service Discovery is the process of automatically
locating a service without knowing its current IP address or physical location.

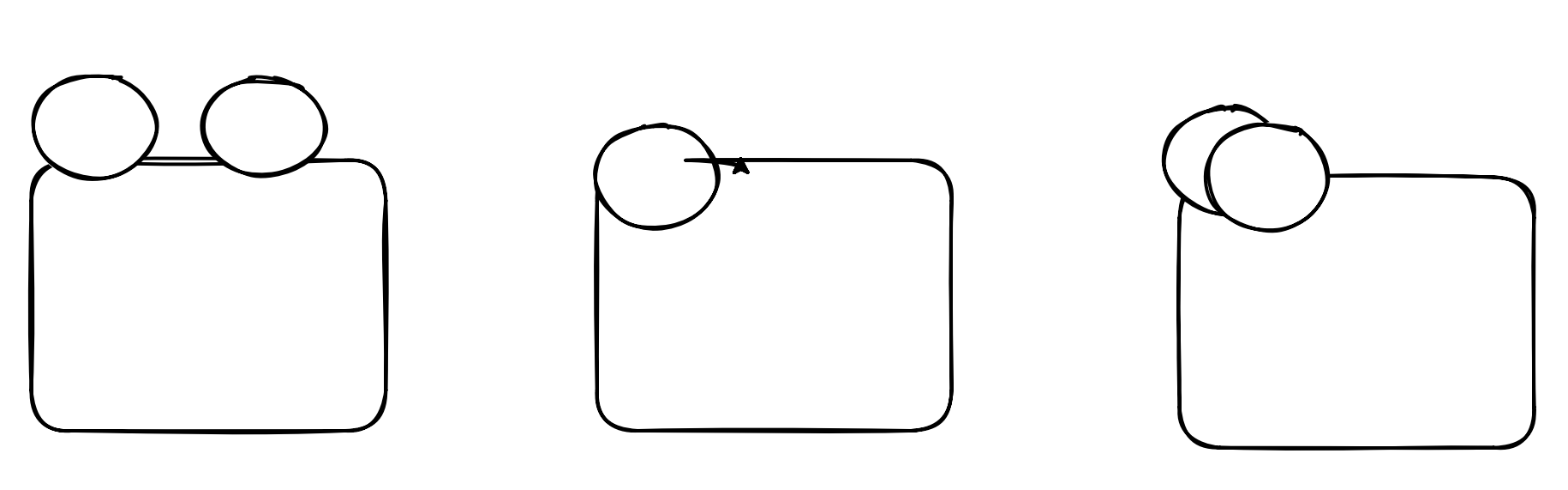
## Self Healing



That ability of the system to automatically recover from failures without human intervention is called Self-Healing.

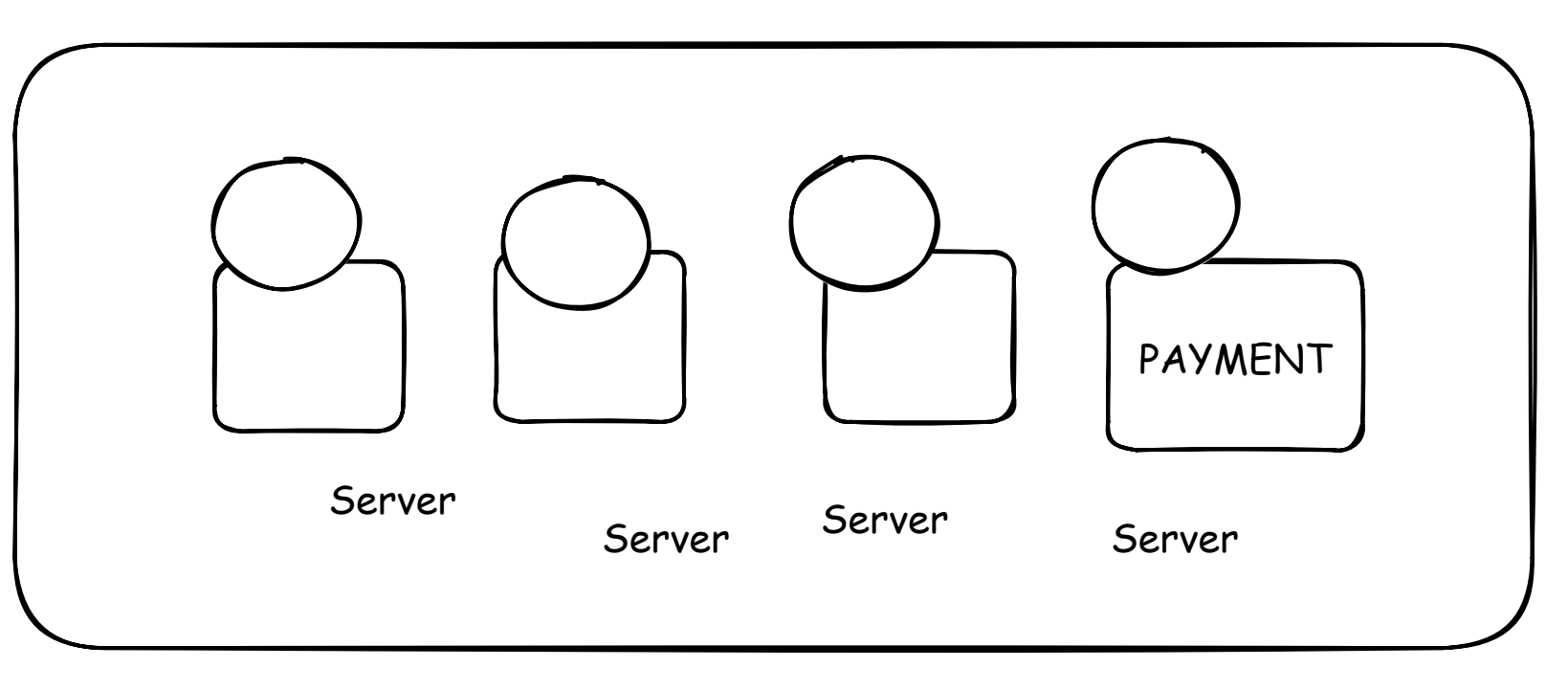
## Fault Tollerance

Ms== 5 conai



Docker orchestration tool
↓
Docker SWARM

Cluster



A CLuster of 4 servers
Node

Amazon Prime Day